

Diffusion Policy · arXiv 2303.04137

Archaeologist

11+3 · seat 1 = Kanta

Priors → DP → flow → RTC → BEHAVIOR
chunk-commit-replan habit

Sep 21 paper club · VLA Architectures

Spine: chunk-commit-replan (T_p / T_a) — NOT classical planner

Chi et al. · RSS 2023 / IJRR extended

Talk arc · one-slide narrative

Archaeologist through-line for Sep 21



HONEST READING

The habit of the control loop persisted.
The denoiser changed (DDPM → flow → soft-inpaint + Σ).

Do NOT claim BEHAVIOR trains Diffusion Policy itself — they train flow on $\pi_{0.5}$.

Influential priors · what Diffusion Policy absorbs

Archaeologist · densest slide — speak the FOILS, not every citation

	PRIOR	CITE	ROLE
	DDPM / score / Langevin	Ho'20 · Song'19 · Welling	Generative denoising backbone
	DDIM / iDDPM	Song'21 · Nichol'21	Fast sampling for real-time control
	Diffuser (FOIL)	Janner'22	Trajectory diffusion — DP = actions-only + O-cond
	IBC (FOIL)	Florence'21	Implicit BC — unstable / slow InfoNCE
	BC-RNN / Robomimic / BET	Mandlekar'21 · Shafiullah'22	Multimodal BC baselines
	FiLM / minGPT	Perez'18 · Karpathy	Conditioning / Transformer pieces
	Mayne RHC	Mayne & Michalska'88	Classical receding-horizon ancestry
	Relay Kitchen	Gupta'19	Long-horizon multitask benchmark

Diffuser foil · planning vs control

Janner et al. 2022 vs Chi et al. 2023 — same family, different contract

Diffuser / planning diffusion

- Diffuse joint states + actions
- Long-horizon trajectory plans
- Often open-loop / rare replan
- Explicit dynamics / cost / goals
- Global trajectory consistency

Diffusion Policy (control)

- Diffuse actions ONLY
- Condition on current O_t
- Receding $T_a < T_p$ replan
- Implicit dynamics in denoiser
- Local multimodal $p(A|O)$

§4.5: linear plant + linear demo → optimal denoiser recovers feedback;
multistep $T_p > 1 \Rightarrow$ learner must implicitly learn task dynamics — controller synthesis via BC, not shoot-through planning.

Contemporaries · same era, parallel tracks

ACT

Zhao et al. · ALOHA

CVAE + Transformer chunks
Temporal ensembling
Parallel chunking invention
(NOT derived from DP)

Other diffusion policies

Pearce · Reuss · IDQL

Sim / RL / goal-cond.
Paper cites as complementary
Same score/flow family

DP differentiator

Chi et al. 2023

Visuomotor real-robot
Receding-horizon + FiLM
Position synergy + 15-task
breadth

Do not fabricate: DP did not invent ACT chunking (parallel).

Shared idea: predict a chunk, execute a prefix, keep temporal consistency.

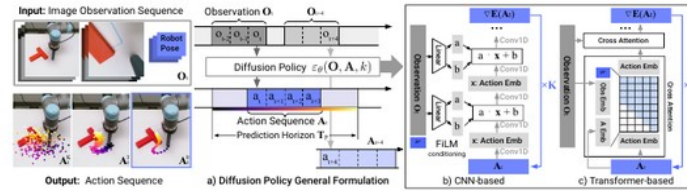


Figure 2. Diffusion Policy Overview a) General formulation. At time step t , the policy takes the latest T_o steps of observation data O_t as input and outputs T_p steps of actions A_t . b) In the CNN-based Diffusion Policy, FILM (Feature-wise Linear Modulation) Perez et al. (2018) conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from A_t^k drawn from Gaussian noise, the output of noise-prediction network ϵ_θ is subtracted, repeating K times to get A_t^k , the denoised action sequence. c) In the Transformer-based Vaswani et al. (2017) Diffusion Policy, the embedding of observation O_t is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated.

2.2 DDPM Training

The training process starts by randomly drawing unmodified examples, x^0 , from the dataset. For each sample, we randomly select a denoising iteration k and then sample a random noise ϵ^k with appropriate variance for iteration k . The noise prediction network is asked to predict the noise from the data sample with noise added.

$$\mathcal{L} = \text{MSE}(\epsilon^k, \epsilon_\theta(x^0 + \epsilon^k, k)) \quad (3)$$

As shown in Ho et al. (2020), minimizing the loss function in Eq 3 also minimizes the variational lower bound of the KL-divergence between the data distribution $p(x^0)$ and the distribution of samples drawn from the DDPM $q(x^0)$ using Eq 1.

2.3 Diffusion for Visuomotor Policy Learning

While DDPMs are typically used for image generation (x is an image), we use a DDPM to learn robot visuomotor policies. This requires two major modifications in the formulation: 1. changing the output x to represent robot actions. 2. making the denoising processes *conditioned* on input observation O_t . The following paragraphs discuss each of the modifications, and Fig. 2 shows an overview.

Closed-loop action-sequence prediction: An effective action formulation should encourage temporal consistency and smoothness in long-horizon planning while allowing prompt reactions to unexpected observations. To accomplish this goal, we commit to the action-sequence prediction produced by a diffusion model for a fixed duration before replanning. Concretely, at time step t the policy takes the latest T_o steps of observation data O_t as input and predicts T_p steps of actions, of which T_a steps of actions are executed on the robot without re-planning. Here, we define T_o as the observation horizon, T_p as the action prediction horizon and T_a as the action execution horizon. This encourages temporal action consistency while remaining responsive. More details about the effects of T_a are discussed in Sec 4.3. Our formulation also allows receding horizon control Mayne and Michalska (1988) to further improve action smoothness by warm-starting the next inference setup with previous action sequence prediction.

Visual observation conditioning: We use a DDPM to approximate the conditional distribution $p(A_t|O_t)$ instead of the joint distribution $p(A_t, O_t)$ used in Janner et al. (2022a) for planning. This formulation allows the model to predict actions conditioned on observations without the cost of inferring future states, speeding up the diffusion process and improving the accuracy of generated actions. To capture the conditional distribution $p(A_t|O_t)$, we modify Eq 1 to:

$$A_t^{k-1} = \alpha(A_t^k - \gamma \epsilon_\theta(O_t, A_t^k, k) + \mathcal{N}(0, \sigma^2 t)) \quad (4)$$

The training loss is modified from Eq 3 to:

$$\mathcal{L} = \text{MSE}(\epsilon^k, \epsilon_\theta(O_t, A_t^0 + \epsilon^k, k)) \quad (5)$$

The exclusion of observation features O_t from the output of the denoising process significantly improves inference speed and better accommodates real-time control. It also helps to make **end-to-end** training of the vision encoder feasible. Details about the visual encoder are described in Sec. 3.2.

3 Key Design Decisions

In this section, we describe key design decisions for Diffusion Policy as well as its concrete implementation of ϵ_θ with neural network architectures.

3.1 Network Architecture Options

The first design decision is the choice of neural network architectures for ϵ_θ . In this work, we examine two common network architecture types, convolutional neural networks (CNNs) Ronneberger et al. (2015) and Transformers Vaswani et al. (2017), and compare their performance and training characteristics. Note that the choice of noise prediction network ϵ_θ is independent of visual encoders, which will be described in Sec. 3.2.

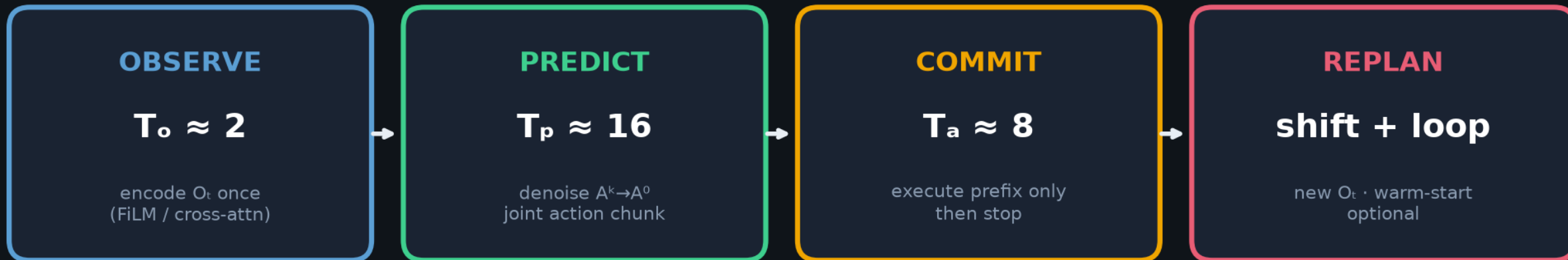
CNN-based Diffusion Policy We adopt the 1D temporal CNN from Janner et al. (2022b) with a few modifications: First, we only model the conditional distribution $p(A_t|O_t)$ by conditioning the action generation process on observation features O_t with Feature-wise Linear Modulation (FILM)

Fig 2 · horizons $T_o/T_p/T_a$ + CNN/Transformer

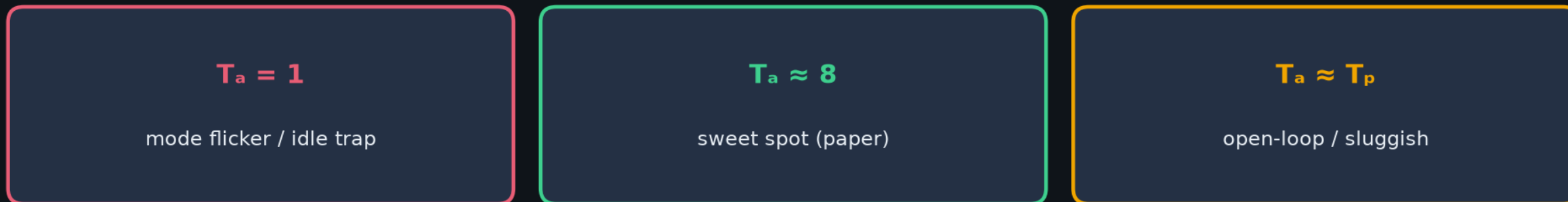
arXiv 2303.04137

Chunk-Commit-Replan · the motion/control contract

NOT classical planner / Diffuser full-trajectory planning · actions-only, O-conditioned



One control cycle (@ ~10 Hz cmds → 125 Hz interp)



Multimodal basins · Langevin commits, then holds

Push-T left/right · Kitchen order · BlockPush stage order



Diffusion Policy

commits to ONE basin per rollout

temporal consistency across T_p

BET / BC-RNN (foil)

mode flicker mid-horizon

independent per-step GMM/k-means

Perez et al. (2018) as well as denoising iteration k , shown in Fig 2 (b). Second, we only predict the action trajectory instead of the concatenated observation action trajectory. Third, we removed inpainting-based goal state conditioning due to incompatibility with our framework utilizing a receding prediction horizon. However, goal conditioning is still possible with the same FiLM conditioning method used for observations.

In practice, we found the CNN-based backbone to work well on most tasks out of the box without the need for much hyperparameter tuning. However, it performs poorly when the desired action sequence changes quickly and sharply through time (such as velocity command action space), likely due to the inductive bias of temporal convolutions to prefer low-frequency signals Tancik et al. (2020).

Time-series diffusion transformer To reduce the over-smoothing effect in CNN models Tancik et al. (2020), we introduce a novel transformer-based DDPM which adopts the transformer architecture from minGPT Shafullah et al. (2022) for action prediction. Actions with noise A_t^k are passed in as input tokens for the transformer decoder blocks, with the sinusoidal embedding for diffusion iteration k prepended as the first token. The observation O_t is transformed into observation embedding sequence by a shared MLP, which is then passed into the transformer decoder stack as input features. The “gradient” $e_\theta(O_t, A_t^k, k)$ is predicted by each corresponding output token of the decoder stack.

In our state-based experiments, most of the best-performing policies are achieved with the transformer backbone, especially when the task complexity and rate of action change are high. However, we found the transformer to be more sensitive to hyperparameters. The difficulty of transformer training Liu et al. (2020) is not unique to Diffusion Policy and could potentially be resolved in the future with improved transformer training techniques or increased data scale.

Recommendations. In general, we recommend starting with the CNN-based diffusion policy implementation as the first attempt at a new task. If performance is low due to task complexity or high-rate action changes, then the Time-series Diffusion Transformer formulation can be used to potentially improve performance at the cost of additional tuning.

3.2 Visual Encoder

The visual encoder maps the raw image sequence into a latent embedding O_t and is trained end-to-end with the diffusion policy. Different camera views use separate encoders, and images in each timestep are encoded independently and then concatenated to form O_t . We used a standard ResNet-18 (without pretraining) as the encoder with the following modifications: 1) Replace the global average pooling with a spatial softmax pooling to maintain spatial information Mandlekar et al. (2021). 2) Replace BatchNorm with GroupNorm Wu and He (2018) for stable training. This is important when the normalization layer is used in conjunction with Exponential Moving Average He et al. (2020) (commonly used in DDPMs).

3.3 Noise Schedule

The noise schedule, defined by σ , α , γ and the additive Gaussian Noise ϵ^k as functions of k , has been actively studied Ho et al. (2020); Nichol and Dhariwal (2021). The underlying noise schedule controls the extent to which diffusion policy captures high and low-frequency characteristics of action signals. In our control tasks, we empirically found that the Square Cosine Schedule proposed in iDDPM Nichol and Dhariwal (2021) works best for our tasks.

3.4 Accelerating Inference for Real-time Control

We use the diffusion process as the policy for robots; hence, it is critical to have a fast inference speed for closed-loop real-time control. The Denoising Diffusion Implicit Models (DDIM) approach Song et al. (2021) decouples the number of denoising iterations in training and inference, thereby allowing the algorithm to use fewer iterations for inference to speed up the process. In our real-world experiments, using DDIM with 100 training iterations and 10 inference iterations enables 0.1s inference latency on a Nvidia 3080 GPU.

4 Intriguing Properties of Diffusion Policy

In this section, we provide some insights and intuitions about diffusion policy and its advantages over other forms of policy representations.

4.1 Model Multi-Modal Action Distributions

The challenge of modeling multi-modal distribution in human demonstrations has been widely discussed in behavior cloning literature Florence et al. (2021); Shafullah et al. (2022); Mandlekar et al. (2021). Diffusion Policy’s ability to express multimodal distributions naturally and precisely is one of its key advantages.

Intuitively, multi-modality in action generation for diffusion policy arises from two sources – an underlying stochastic sampling procedure and a stochastic initialization. In Stochastic Langevin Dynamics, an initial sample A_t^0 is drawn from standard Gaussian at the beginning of each sampling process, which helps specify different possible convergence basins for the final action prediction A_t^0 . This action is then further stochastically optimized, with added Gaussian perturbations across a large number of iterations, which enables individual action samples to converge and move between different multi-modal action basins. Fig. 3, shows an example of the Diffusion Policy’s multimodal behavior in a planar pushing task (Push T, introduced below) without explicit demonstration for the tested scenario.

4.2 Synergy with Position Control

We find that Diffusion Policy with a position-control action space consistently outperforms Diffusion Policy with velocity control, as shown in Fig 4. This surprising result stands in contrast to the majority of recent behavior cloning work that generally relies on velocity control Mandlekar et al. (2021); Shafullah et al. (2022); Zhang et al. (2018); Florence et al. (2019); Mandlekar et al. (2020b,a). We

Fig 3 · multimodal Push-T basins

arXiv 2303.04137

Subsequent lineage · CLEAR citations only

Denoiser changed · chunk-commit-replan-overlap habit stayed

2023

**Diffusion
Policy**



DDPM chunks
 T_p/T_a replan

2023

ACT



CVAE chunks
(parallel)

2024

π_0



cites Chi'23
DDPM→flow

2025

**$\pi_{0.5}$ /
OpenPI**



same chunk
habit

2025

**RTC +
BEHAVIOR**



soft-inpaint
30/26/4

π_0 · 2410.24164

- Cites Chi et al. 2023
- Flow matching on PaliGemma
- Continuous chunks ~50 Hz
- Same control ontology

RTC · 2506.07339

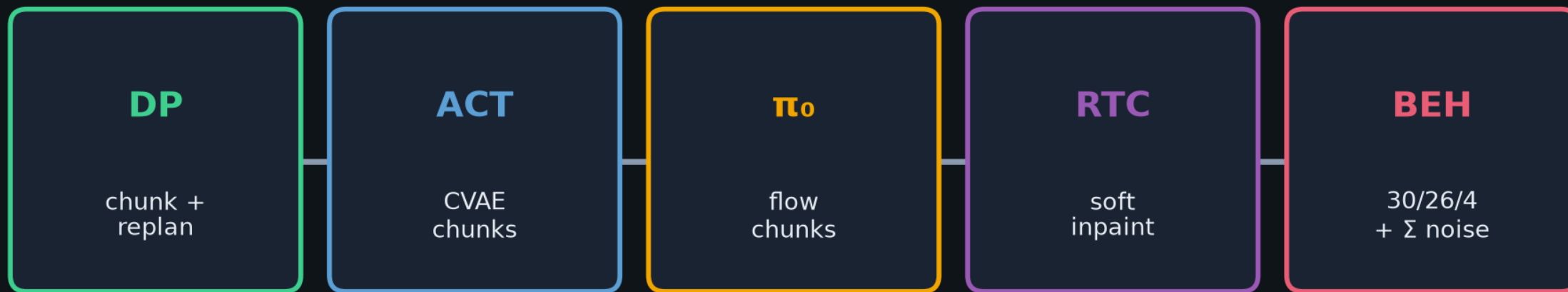
- Soft-inpaint async chunks
- Freeze delay-safe prefix
- Works on diffusion OR flow
- No retrain

BEHAVIOR 1st · 2512.06951

- Pi0.5 + corr. noise
- predict 30 / exec 26 / inpaint 4
- $q \sim 0.26$
- NOT training DP itself

Club line to BEHAVIOR

DP chunk+replan → ACT → π_0 flow chunks → RTC soft-inpaint → BEHAVIOR (exec26/inpaint4)



What stayed vs what moved

STAYED

- generative continuous chunks
 - execute-prefix / replan
- overlap consistency need

MOVED

- DDPM → flow matching
- sync → async soft-inpaint
- implicit corr → explicit Σ

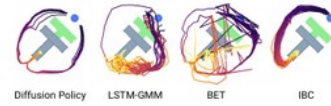


Figure 3. Multimodal behavior. At the given state, the end-effector (blue) can either go left or right to push the block. **Diffusion Policy** learns both modes and commits to only one mode within each rollout. In contrast, both **LSTM-GMM** Mandelkar et al. (2021) and **IBC** Florence et al. (2021) are biased toward one mode, while **BET** Shafiiullah et al. (2022) fails to commit to a single mode due to its lack of temporal action consistency. Actions generated by rolling out 40 steps for the best-performing checkpoint.

speculate that there are two primary reasons for this discrepancy: First, action multimodality is more pronounced in position-control mode than it is when using velocity control. Because Diffusion Policy better expresses action multimodality than existing approaches, we speculate that it is inherently less affected by this drawback than existing methods. Furthermore, position control suffers less than velocity control from compounding error effects and is thus more suitable for action-sequence prediction (as discussed in the following section). As a result, Diffusion Policy is both less affected by the primary drawbacks of position control and is better able to exploit position control’s advantages.

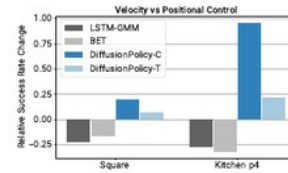


Figure 4. Velocity v.s. Position Control. The performance difference when switching from velocity to position control. While both BCRNN and BET performance decrease, Diffusion Policy is able to leverage the advantage of position and improve its performance.

4.3 Benefits of Action-Sequence Prediction

Sequence prediction is often avoided in most policy learning methods due to the difficulties in effectively sampling from high-dimensional output spaces. For example, IBC would struggle in effectively sampling high-dimensional action space with a non-smooth energy landscape. Similarly, BC-RNN and BET would have difficulty specifying the number of modes that exist in the action distribution (needed for GMM or k-means steps).

In contrast, DDPM scales well with output dimensions without sacrificing the expressiveness of the model, as demonstrated in many image generation applications. Leveraging this capability, Diffusion Policy represents action in the form of a high-dimensional action sequence, which naturally addresses the following issues:

- **Temporal action consistency:** Take Fig 3 as an example. To push the T block into the target from the bottom, the policy can go around the T block from either left or right. However, suppose each action in the sequence is predicted as independent multimodal distributions (as done in BC-RNN and BET). In that case, consecutive actions could be drawn from different modes, resulting in jittery actions that alternate between the two valid trajectories.
- **Robustness to idle actions:** Idle actions occur when a demonstration is paused and results in sequences of identical positional actions or near-zero velocity actions. It is common during teleoperation and is sometimes required for tasks like liquid pouring. However, single-step policies can easily overfit to this pausing behavior. For example, BC-RNN and IBC often get stuck in real-world experiments when the idle actions are not explicitly removed from training.

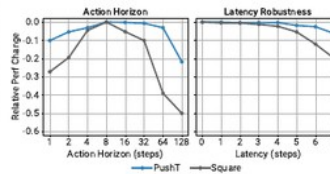


Figure 5. Diffusion Policy Ablation Study. Change (difference) in success rate relative to the maximum for each task is shown on the Y-axis. **Left:** trade-off between temporal consistency and responsiveness when selecting the action horizon. **Right:** Diffusion Policy with position control is robust against latency. Latency is defined as the number of steps between the last frame of observations to the first action that can be executed.

4.4 Training Stability

While IBC, in theory, should possess similar advantages as diffusion policies. However, achieving reliable and high-performance results from IBC in practice is challenging due to IBC’s inherent training instability Ta et al. (2022). Fig 6 shows training error spikes and unstable evaluation performance throughout the training process, making hyperparameter tuning critical and checkpoint selection difficult. As a result, Florence et al. (2021) evaluate every checkpoint and report results for the best-performing checkpoint. In a real-world setting, this workflow necessitates the evaluation of many policies on hardware to select a final policy. Here, we discuss why Diffusion Policy appears significantly more stable to train.

An implicit policy represents the action distribution using an Energy-Based Model (EBM):

$$p_{\theta}(\mathbf{a}|\mathbf{o}) = \frac{e^{-E_{\theta}(\mathbf{o}, \mathbf{a})}}{Z(\mathbf{o}, \theta)} \quad (6)$$

where $Z(\mathbf{o}, \theta)$ is an intractable normalization constant (with respect to $\mathbf{a}$).

Q & A

Habit persisted · denoiser changed

Diffusion Policy · Sep 21

Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

Cheng Chi^{*1}, Zhenjia Xu^{*1}, Siyuan Feng², Eric Cousineau², Yilun Du³, Benjamin Burchfiel², Russ Tedrake^{2,3}, Shuran Song^{1,4}

Abstract

This paper introduces Diffusion Policy, a new way of generating robot behavior by representing a robot's visuomotor policy as a conditional denoising diffusion process. We benchmark Diffusion Policy across 15 different tasks from 4 different robot manipulation benchmarks and find that it consistently outperforms existing state-of-the-art robot learning methods with an average improvement of 46.9%. Diffusion Policy learns the gradient of the action-distribution score function and iteratively optimizes with respect to this gradient field during inference via a series of stochastic Langevin dynamics steps. We find that the diffusion formulation yields powerful advantages when used for robot policies, including gracefully handling multimodal action distributions, being suitable for high-dimensional action spaces, and exhibiting impressive training stability. To fully unlock the potential of diffusion models for visuomotor policy learning on physical robots, this paper presents a set of key technical contributions including the incorporation of receding horizon control, visual conditioning, and the time-series diffusion transformer. We hope this work will help motivate a new generation of policy learning techniques that are able to leverage the powerful generative modeling capabilities of diffusion models. Code, data, and training details is available diffusion-policy.cs.columbia.edu

Keywords

Imitation learning, visuomotor policy, manipulation

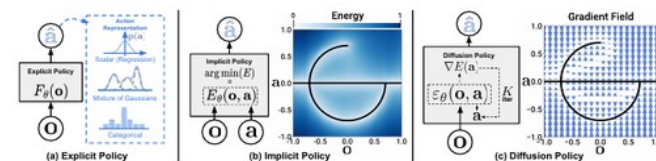


Figure 1. Policy Representations. a) Explicit policy with different types of action representations. b) Implicit policy learns an energy function conditioned on both actions and observation and optimizes for actions that minimize the energy landscape c) Diffusion policy refines noise into actions via a learned gradient field. This formulation provides stable training, allows the learned policy to accurately model multimodal action distributions, and accommodates high-dimensional action sequences.

1 Introduction

Policy learning from demonstration, in its simplest form, can be formulated as the supervised regression task of learning to map observations to actions. In practice however, the unique nature of predicting robot actions — such as the existence of multimodal distributions, sequential correlation, and the requirement of high precision — makes this task distinct and challenging compared to other supervised learning problems.

Prior work attempts to address this challenge by exploring different *action representations* (Fig 1 a) — using mixtures of Gaussians [Mandlekar et al. \(2021\)](#), categorical representations of quantized actions [Shafullah et al. \(2022\)](#), or by switching the *policy representation* (Fig 1 b) — from explicit to implicit to better capture multi-modal distributions [Florence et al. \(2021\)](#); [Wu et al. \(2020\)](#).

In this work, we seek to address this challenge by introducing a new form of robot visuomotor policy that

generates behavior via a “conditional denoising diffusion process” [Ho et al. \(2020\)](#) on robot action space”, **Diffusion Policy**. In this formulation, instead of directly outputting an action, the policy infers the action-score gradient, conditioned on visual observations, for K denoising iterations (Fig. 1 c). This formulation allows robot policies to inherit several key properties from diffusion models — significantly improving performance.

- **Expressing multimodal action distributions.** By learning the gradient of the action score function [Song and Ermon \(2019\)](#) and performing Stochastic Langevin Dynamics sampling on this gradient field, Diffusion policy can express arbitrary normalizable distributions [Neal et al. \(2011\)](#), which includes multimodal action distributions, a well-known challenge for policy learning.